

Конвейер процессора

Повторение: поток управления

Что делают и как работают инструкции:

- **JMP**
- **CALL, RET**
- **CMP, TEST**
- **JAE, JBE, JE, JG, JL** и др.

Повторение: поток управления

Что делают и как работают инструкции:

- **JMP** – безусловный прыжок по метке. Переводит EIP в адрес метки
- **CALL, RET** – CALL кладет текущий адрес (EIP) на стек, прыгает по метке функции. RET – прыгает по адресу со стека.
- **CMP, TEST** – Обе выполняют операцию, выставляют флаги, отбрасывают результат. CMP – вычитание, TEST – побитовый and
- **JAE, JBE, JE, JG, JL** и др. – Условные прыжки. Если условие верно – происходит прыжок, если нет – выполняем следующую инструкцию.

Повторение: метки

Что такое метка в ассемблере?

Повторение: метки

Метка – это способ пометить адрес программы во время ее написания.

Метки заменяются на адреса во время трансляции ассемблера в байткод.

Метки пропадают после ассемблирования программы. **В бинарном файле не существует меток.** Любое использование метки заменяется на соответствующий адрес.

```
0804922e <main>:  
804922e: b8 fe ff ff ff      mov     eax,0xffffffff  
8049233: e8 d5 ff ff ff      call    804920d <run_case>
```

Код после ассемблирования

Повторение: циклы

Давайте транслируем следующий цикл с языка C в NASM

Повторение: циклы

Давайте транслируем следующий цикл с языка C в NASM

```
1 int array[] = {7, -3, 0, 11, -1, 5, -9, 2, 4, -6};
2 const int array_size = (int)(sizeof(array) / sizeof(array[0]));
3
4 static int sum_positive_c(void) {
5     int sum = 0;
6     for (int i = 0; i < array_size; i++) {
7         if (array[i] > 0) {
8             sum += array[i];
9         }
10    }
11    return sum;
12 }
```

Повторение: циклы

Рукописный вариант
выглядит так

```
1 sum_positive_asm:
2     mov esi, array
3     mov ecx, [array_size]
4     mov eax, 0
5
6 .loop:
7     cmp ecx, 0
8     jz .done
9
10    mov edx, [esi]           ; x = *arr
11    cmp edx, 0
12    jle .skip_add           ; if (x <= 0) skip
13    add eax, edx             ; sum += x
14
15 .skip_add:
16    add esi, 4
17    dec ecx
18    jmp .loop
19
20 .done:
21    ret
```

Повторение: граф потока управления

Рукописный вариант
выглядит так

Немного сложно
воспринимать такое
полотно текста.

Давайте разобьем код
на блоки без
ветвлений. А такие
блоки соединим

```
1 sum_positive_asm:
2     mov esi, array
3     mov ecx, [array_size]
4     mov eax, 0
5
6 .loop:
7     cmp ecx, 0
8     jz .done
9
10    mov edx, [esi]           ; x = *arr
11    cmp edx, 0
12    jle .skip_add           ; if (x <= 0) skip
13    add eax, edx             ; sum += x
14
15 .skip_add:
16    add esi, 4
17    dec ecx
18    jmp .loop
19
20 .done:
21    ret
```

Повторение: граф потока управления

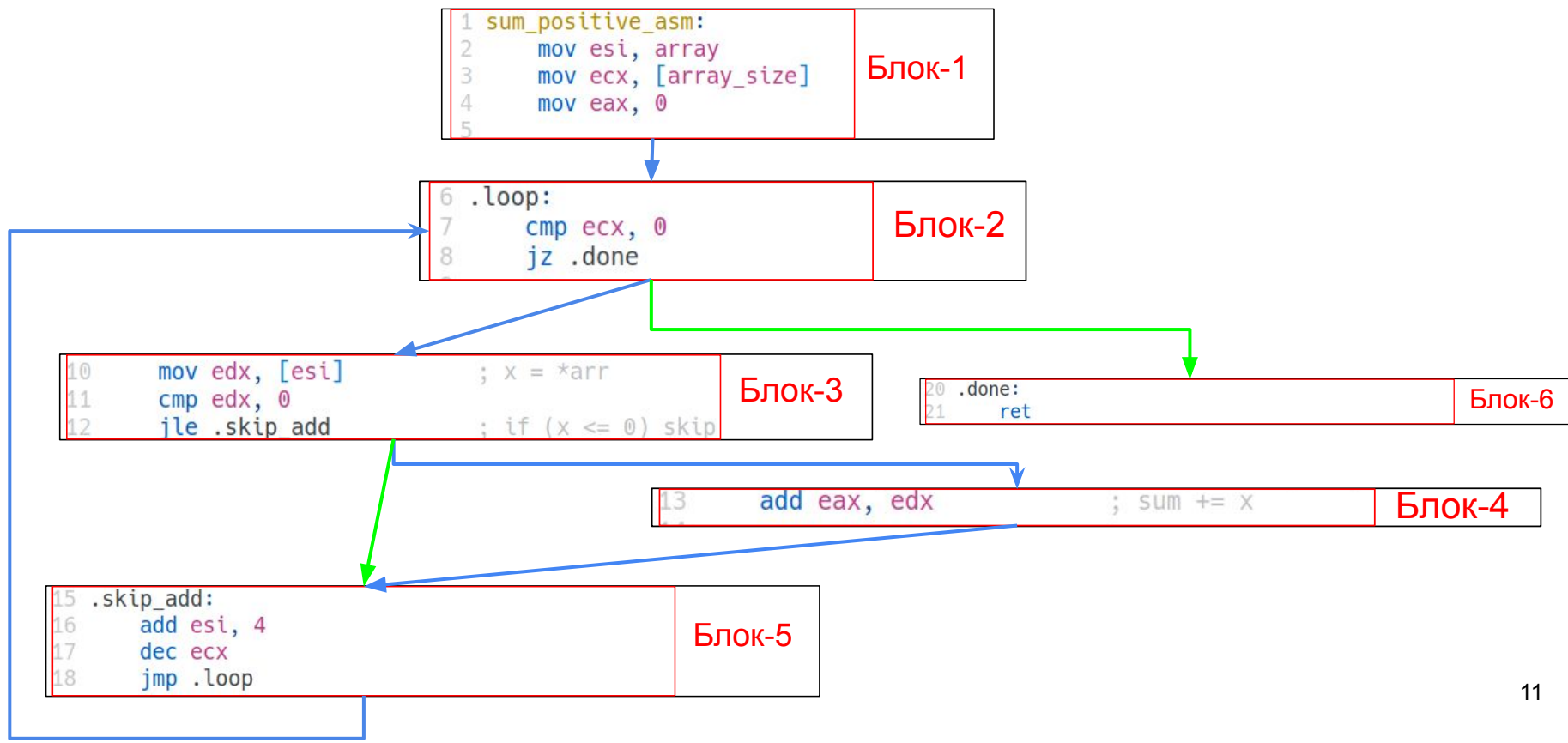
Рукописный вариант
выглядит так

Немного сложно
воспринимать такое
полотно текста.

Давайте разобьем код
на блоки без
ветвлений. А такие
блоки соединим

1 sum_positive_asm: 2 mov esi, array 3 mov ecx, [array_size] 4 mov eax, 0 5	Блок-1
6 .loop: 7 cmp ecx, 0 8 jz .done 9	Блок-2
10 mov edx, [esi] ; x = *arr 11 cmp edx, 0 12 jle .skip_add ; if (x <= 0) skip	Блок-3
13 add eax, edx ; sum += x 14	Блок-4
15 .skip_add: 16 add esi, 4 17 dec ecx 18 jmp .loop 19	Блок-5
20 .done: 21 ret	Блок-6

Повторение: граф потока управления

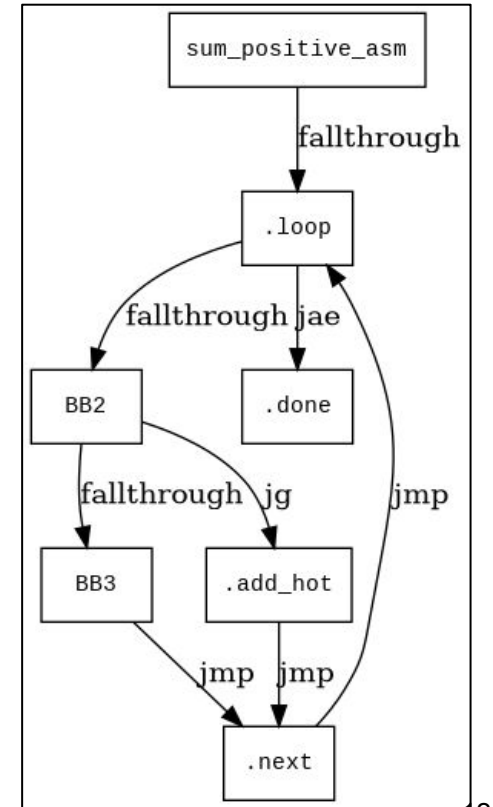


Граф потока управления (нормальный)

.next – инкремент переменной + решение о выходе из цикла

BB2 – решение добавлять элемент к сумме или пропускать сложение

.add_hot – сложение (происходит часто на наших данных)



Граф потока управления (нормальный)

С помощью control-flow graph (CFG) удобно воспринимать код с точки зрения ветвлений, циклов, передачи управления.

Мы уже посмотрели, как выглядит CFG рукописного ассемблерного кода.

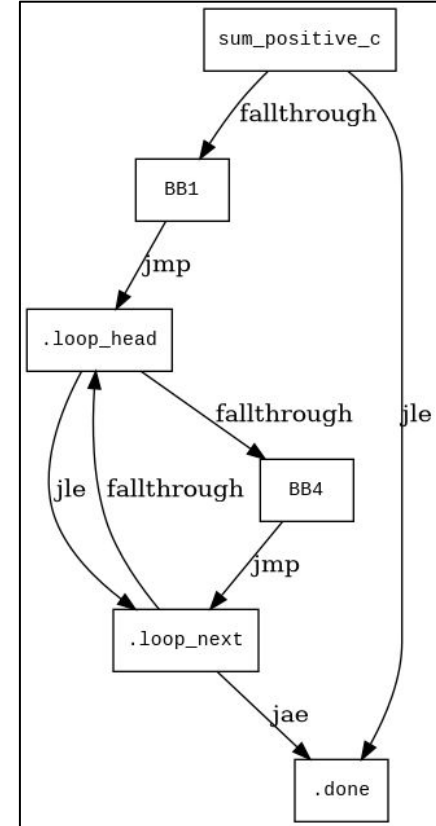
Давайте посмотрим на CFG того же цикла, но теперь ассемблер нам сгенерирует компилятор языка C. То есть напишем программу на языке C,

Граф потока управления (от компилятора)

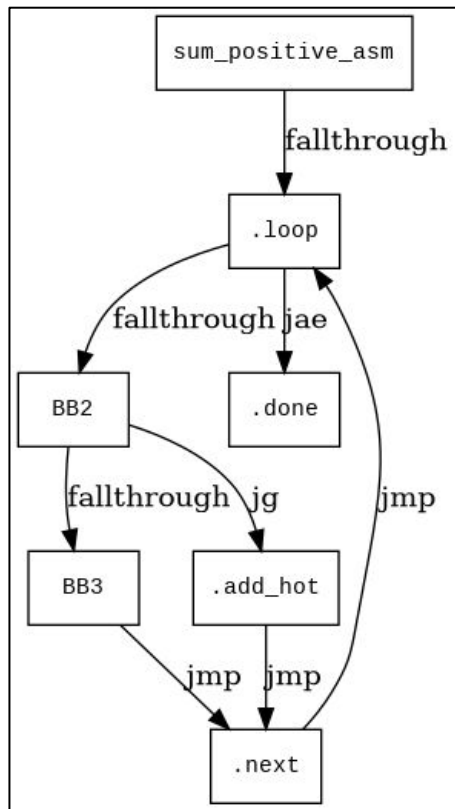
loop_next – инкремент переменной +
решение о выходе из цикла

loop_head – решение добавлять элемент к
сумме или пропускать сложение

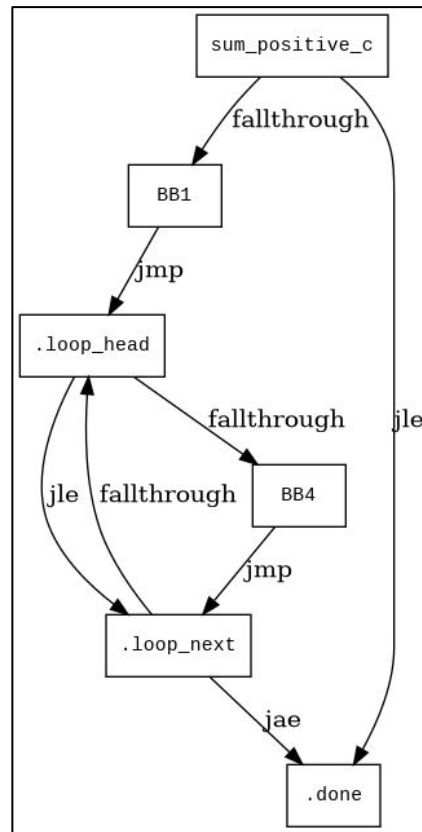
BB4 – сложение (происходит часто на
наших данных)



Графы потока управления (сравнение)



человек



бездушная
машина
(компилятор)

Причина различий

- Почему компилятор генерирует граф отличный от нашего?
- Видимо он что-то знает про процессор, чего не знаем мы.

Причина различий

- Почему компилятор генерирует граф отличный от нашего?
- Видимо он что-то знает про процессор, чего не знаем мы.
- Компилятор знает про конвейер процессора или CPU pipeline. Давайте разбираться, что это

CPU Pipeline

Каждая инструкция исполняется не “мгновенно”, а по шагам. Шаги всегда одни и те же:

1. **FE – Fetch** – получить байткод инструкции из памяти
2. **DE – Decode** – понять, что за инструкция перед нами, какие у нее операнды, считать нужные регистры
3. **EX – Execute** – выполнить арифметико-логические операции, определить результат сравнения для ветвлений
4. **ME – Memory Access** – при необходимости читаем и записываем в память
5. **WB – WriteBack** – результат исполнения записывается в регистр назначения

CPU Pipeline

1. **FE – Fetch** – получить байткод инструкции из памяти
2. **DE – Decode** – понять, что за инструкция перед нами, какие у нее операнды, считать нужные регистры
3. **EX – Execute** – выполнить арифметико-логические операции, определить результат сравнения для ветвлений
4. **ME – Memory Access** – при необходимости читаем и записываем в память
5. **WB – WriteBack** – результат исполнения записывается в регистр назначения

За счет разбиения на шаги, мы можем ввести параллелизм уровня инструкций!

CPU Pipeline

За счет разбиения на шаги, мы можем ввести параллелизм уровня инструкций!

Пока одна инструкция исполняется на шаге 2 (DE), следующая за ней может не ждать полного выполнения, а начать исполняться на шаге 1 (FE).

То есть процессор считывает следующую инструкцию еще до того, как закончил исполнять текущую.

При хороших условиях, пайплайн полностью заполнен. То есть на каждом этапе исполняется инструкция. Но так бывает не всегда. Почему?

CPU Pipeline. Зависимости

При хороших условиях, пайплайн полностью заполнен. То есть на каждом этапе выполняется инструкция. Но так бывает не всегда. Почему?

Дело в том, что инструкции могут зависеть друг от друга. Если следующая инструкция использует регистр, в который пишется результат текущей инструкции, мы обязаны дождаться конца исполнения.

Эту проблему решает компилятор, перемещая инструкции так, чтоб они максимально не зависели друг от друга.

Но есть и другая проблема

CPU Pipeline. Циклы

Есть и другая проблема. Код и пайплайн на слайде. Какую инструкцию грузить (FE) следующей?

```
1 foo:
2   cmp eax, 1
3   jg .finish:
4   add eax, 1
5
6 .finish:
7   sub eax, 1
8   ret
9
```

FE	DE	EX	ME	WB	
???	jg .finish	...	...	...	

CPU Pipeline. Циклы

Есть и другая проблема. Код и пайплайн на слайде. Какую инструкцию грузить (FE) следующей? Это зависит от того, прыгнем мы, или нет. Как узнать это заранее?

```
1 foo:
2   cmp eax, 1
3   jg .finish:
4   add eax, 1
5
6 .finish:
7   sub eax, 1
8   ret
9
```



Если переход не взят



Если переход взят

CPU Pipeline. Branch-predictor

В процессоре есть специальное устройство, которое предсказывает переходы. Оно довольно умное и умеет учиться на статистике.

```
1 foo:  
2   cmp eax, 1  
3   jg .finish:  
4   add eax, 1  
5  
6 .finish:  
7   sub eax, 1  
8   ret  
9
```



Если переход не взят



Если переход взят

CPU Pipeline. Branch-predictor

Итого, правильный ответ: “зависит от статистики выполнения foo”. Если чаще всего переход брался, то загрузится sub. Если чаще нет – загрузится add

```
1 foo:
2   cmp eax, 1
3   jg .finish:
4   add eax, 1
5
6 .finish:
7   sub eax, 1
8   ret
9
```



Если переход не взят



Если переход взят

CPU Pipeline. Branch-predictor

```
1 foo:
2   cmp eax, 1
3   jg .finish:
4   add eax, 1
5
6 .finish:
7   sub eax, 1
8   ret
9
```

А если foo выполняется впервые? Тогда по умолчанию branch-predictor считает:

1. если это прыжок назад (как в цикле) – считается, что переход происходит
2. если это прыжок вперед – считается, что переход не происходит

То есть в нашем случае, при первом исполнении foo будет считаться что переход не выполняется.

CPU Pipeline. Branch-predictor

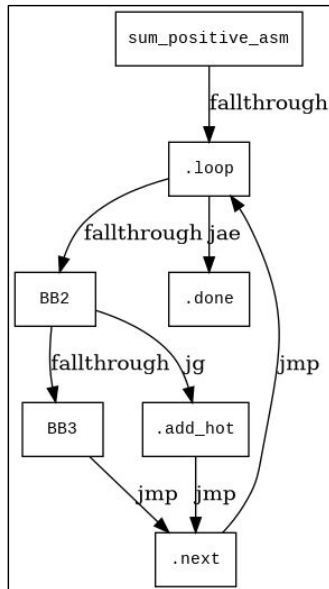
Что если branch-predictor не угадал?

Катастрофы не происходит. Мы *могли бы* исполнить инструкции параллельно, но мы этого не сделали. Нам придется исполнять их последовательно. Ситуация, когда branch-predictor не угадал, называется “branch-miss”.

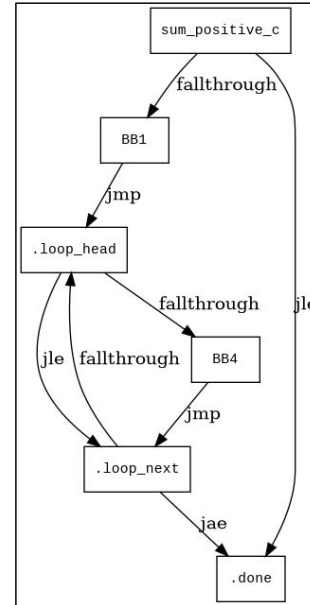
Дальше мы посмотрим, как замерять бранч-мисы и что скорость работы программы существенно зависит от их количества.

CPU Pipeline. Branch-predictor

Перед тем как перейти к экспериментам, вернемся к вопросу про CFG. Почему же компилятор сгенерировал цикл именно так?



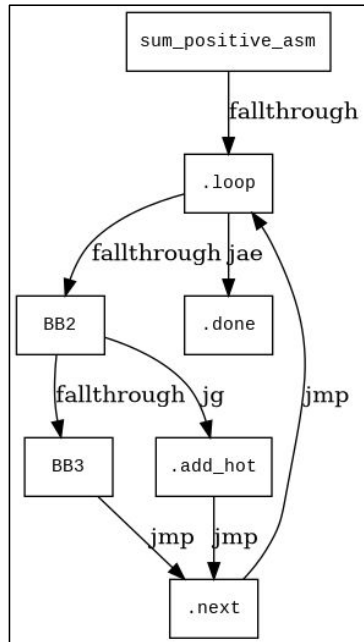
человек



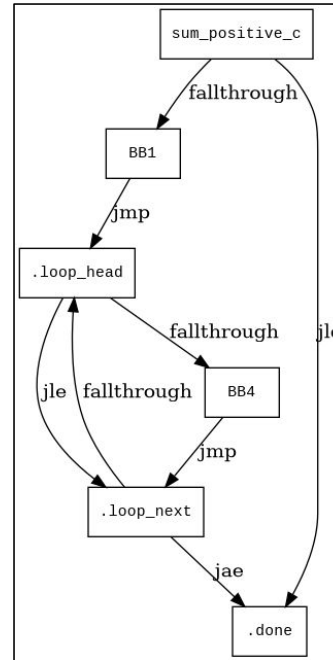
бездушная
машина
(компилятор)

CPU Pipeline. Branch-predictor

Компилятор учел поведение бранч-предиктора. А мы не учли. Компилятор поставил “fallthrough” там где переход вниз берется чаще, потому что так предскажет предиктор



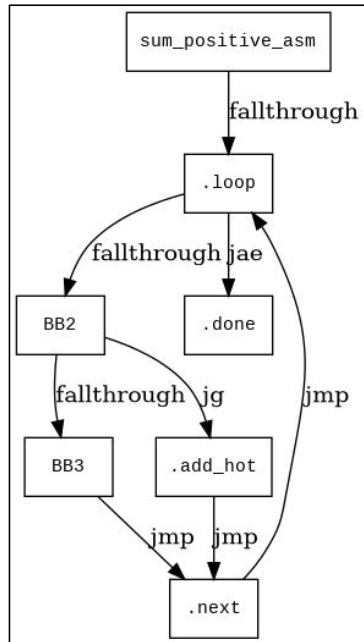
человек



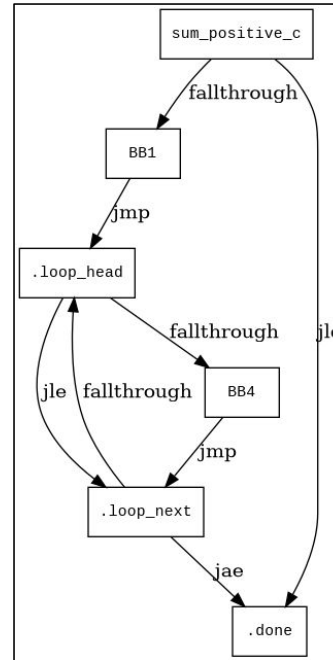
бездушная
машина
(компилятор)

CPU Pipeline. Branch-predictor

Таким образом, код компилятора будет эффективнее, за счет меньшего количества бранч-мисов. Но насколько эффективнее?



человек



бездушная
машина
(компилятор)

CPU Pipeline. Branch-predictor. Эффективность

Рассмотрим цикл как на картинке. Запустим программу в двух вариантах:

1. на массиве только с положительными числами
2. на массиве с числами обоих знаков

Будет ли отличие во времени работы программы?

```
2 static int branchy_sum(const int *arr, int n) {  
3     int sum = 0;  
4     for (int i = 0; i < n; i++) {  
5         if (arr[i] > 0) {  
6             sum += arr[i];  
7             sink = sum; /* keep branch explicit, avoid cmov */  
8         }  
9     }  
10    return sum;  
11 }
```

CPU Pipeline. Branch-predictor. Эффективность

Рассмотрим цикл как на картинке. Запустим программу в двух вариантах:

1. на массиве только с положительными числами
2. на массиве с числами обоих знаков

Будет ли отличие во времени работы программы? Звучит дико, но да!

```
2 static int branchy_sum(const int *arr, int n) {  
3     int sum = 0;  
4     for (int i = 0; i < n; i++) {  
5         if (arr[i] > 0) {  
6             sum += arr[i];  
7             sink = sum; /* keep branch explicit, avoid cmov */  
8         }  
9     }  
10    return sum;  
11 }
```

CPU Pipeline. Branch-predictor. Эффективность

```
=== random ===
mode=random n=20000000 rounds=40 checksum=39993920 sink=999848

Performance counter stats for '/home/danny/research/seminars-on-asm/seminar-6/bin/ex02_branch_predictor random 20000000 40':

    246,303,397      branches
    41,444,252      branch-misses                # 16.83% of all branches
  1,295,225,200      cycles
    803,003,598      instructions                # 0.62 insn per cycle

    0.321851757 seconds time elapsed

    0.317889000 seconds user
    0.003998000 seconds sys
```

Запуск на случайных данных. Мы не угадали почти каждый 6й переход (16%).

CPU Pipeline. Branch-predictor. Эффективность

```
=== predictable ===
mode=predictable n=2000000 rounds=40 checksum=4039615360 sink=100990384

Performance counter stats for '/home/danny/research/seminars-on-asm/seminar-6/bin/ex02_branch_predictor predictable 2000000 40':

   286,865,961      branches
   377,102         branch-misses          #    0.13% of all branches
  305,874,486      cycles
  933,926,046      instructions          #    3.05  insn per cycle

   0.077549448 seconds time elapsed

   0.069518000 seconds user
   0.008060000 seconds sys
```

Запуск на положительных данных.
В 10 раз меньше бранч-мисов. В 50 раз быстрее.

CPU Pipeline. Branch-predictor. Эффективность

Это все конечно хорошо. Но что на практике? Мы же не можем запускать наши программы только на положительных данных. Что делать если мы знаем что наш **if** будет обычно исполняться?

CPU Pipeline. Branch-predictor. Эффективность

Это все конечно хорошо. Но что на практике? Мы же не можем запускать наши программы только на положительных данных. Что делать если мы знаем что наш **if** будет обычно исполняться?

На помощь приходит компиляторная магия!

__builtin_expect(COND, EXPECTED) позволяет нам сказать компилятору, что мы ожидаем, что условие COND примет значение EXPECTED. А компилятор расположит код так, чтоб это соответствовало догадкам бранч-предиктора

Чаще всего пользуются этим через такие макросы

```
#define likely(x)      __builtin_expect(!!(x), 1)
#define unlikely(x)    __builtin_expect(!!(x), 0)
```

CPU Pipeline. Branch-predictor. Эффективность

`__builtin_expect(COND, EXPECTED)` позволяет нам **сказать компилятору**, что мы ожидаем, что условие COND примет значение EXPECTED. А компилятор расположит код так, чтоб это соответствовало дефолтным догадкам бранч-предиктора

Чаще всего пользуются этим через такие макросы

```
3 #define likely(x)      __builtin_expect(!!(x), 1)
4 #define unlikely(x)    __builtin_expect(!!(x), 0)
5
6 static int branchy_sum(const int *arr, int n) {
7     int sum = 0;
8     for (int i = 0; i < n; i++) {
9         if (unlikely(arr[i] > 0)) { // We tell branch-predictor that
10                                     // this condition is unlikely to happen
11             sum += arr[i];
12             sink = sum; /* keep branch explicit, avoid cmov */
13         }
14     }
15     return sum;
16 }
```

Ленивые вычисления

Обсудив сложные вещи, самое время облениться

Все помнят, как в С вычисляются сложные условия?

Что если **p == NULL**?

```
2 static int lazy_chain(const Node *p) {  
3     if (p && p->next != NULL && (p->next->value > 0)) {  
4         return 1;  
5     }  
6     return 0;  
7 }
```

Ленивые вычисления

Обсудив сложные вещи, самое время облениться

Все помнят, как в С вычисляются сложные условия?

Что если **p == NULL**?

Тогда второе и третье условие **не будут вычисляться** и мы не словим сегфолт

```
2 static int lazy_chain(const Node *p) {  
3     if (p && p->next != NULL && (p->next->value > 0)) {  
4         return 1;  
5     }  
6     return 0;  
7 }
```

Ленивые вычисления

Это называется “ленивые вычисления”. Следующее условие вычисляется только при необходимости.

Если первое условие оператора && ложно, остальные не вычисляются

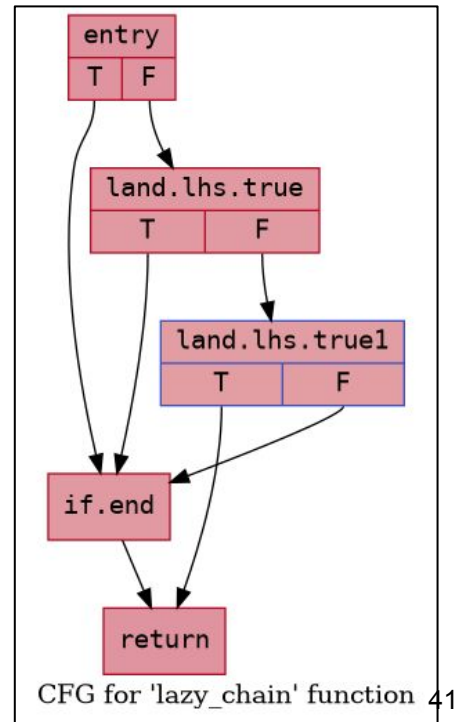
Если первое условие оператора || истинно, остальные не вычисляются

```
2 static int lazy_chain(const Node *p) {  
3     if (p && p->next != NULL && (p->next->value > 0)) {  
4         return 1;  
5     }  
6     return 0;  
7 }
```

Ленивые вычисления

Такое поведение достигается за счет действий компилятора. Он компилирует сложное условие в несколько прыжков.

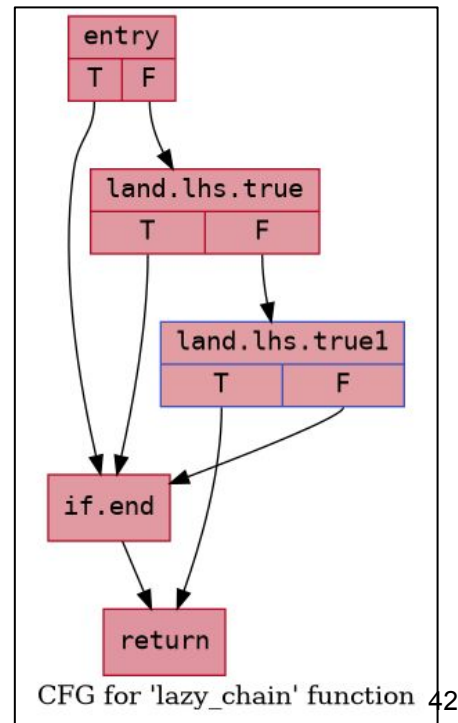
```
2 static int lazy_chain(const Node *p) {  
3     if (p && p->next != NULL && (p->next->value > 0)) {  
4         return 1;  
5     }  
6     return 0;  
7 }
```



Ленивые вычисления

Рассмотрим ассемблер, который нам сгенерирует компилятор из этой функции

```
2 static int lazy_chain(const Node *p) {  
3     if (p && p->next != NULL && (p->next->value > 0)) {  
4         return 1;  
5     }  
6     return 0;  
7 }
```



Ленивые вычисления

Если по какой-то причине надо избежать такого поведения, в С можно использовать побитовые операторы “&” и “|”.

В случае логических выражений, они поведут себя как && и ||, за исключением ленивости.

```
1 int eager_chain(void) {  
2     if ((int)(g_p != NULL) &  
3         (second_checks++, (g_p != NULL ? g_p->next != NULL : 0)) &  
4         (third_checks++, (g_p != NULL && g_p->next != NULL ? g_p->next->value > 0 : 0))) {  
5         return 1;  
6     }  
7     return 0;  
8 }
```

Вопросы